

Unflatten#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.flatten.Unflatten>

Description:

Unflattens a tensor dim expanding it to a desired shape. For use with Sequential. dim specifies the dimension of the input tensor to be unflattened, and it can be either int or str when Tensor or NamedTensor is used, respectively. unflattened_size is the new shape of the unflattened dimension of the tensor and it can be a tuple of ints or a list of ints or torch.Size for Tensor input; a NamedShape (tuple of (name, size) tuples) for NamedTensor input. Input: $(*, S_{\text{dim}}, *)$ $(*, S_{\text{dim}}, *)$, where S_{dim} is the size at dimension dim and $*$ means any number of dimensions including none. Output: $(*, U_1, \dots, U_n, *)$ $(*, U_1, \dots, U_n, *)$, where $U_i = \text{unflattened_size}$ and $\prod_{i=1}^n U_i = S_{\text{dim}}$.

Function Signature

```
class torch.nn.modules.flatten.Unflatten(dim,  
unflattened_size)[source]#
```

Shape:

Parameters

NamedShape#

extra_repr()[source]#

Returns:

Tensor

Code Examples

```
>>> input = torch.randn(2, 50)
>>> # With tuple of ints
>>> m = nn.Sequential(
>>>     nn.Linear(50, 50),
>>>     nn.Unflatten(1, (2, 5, 5))
>>> )
>>> output = m(input)
>>> output.size()
torch.Size([2, 2, 5, 5])
>>> # With torch.Size
>>> m = nn.Sequential(
>>>     nn.Linear(50, 50),
>>>     nn.Unflatten(1, torch.Size([2, 5, 5])))
>>> )
>>> output = m(input)
>>> output.size()
torch.Size([2, 2, 5, 5])
>>> # With namedshape (tuple of tuples)
>>> input = torch.randn(2, 50, names=("N", "features"))
>>> unflatten = nn.Unflatten("features", (("C", 2), ("H", 5),
>>> ("W", 5)))
>>> output = unflatten(input)
>>> output.size()
torch.Size([2, 2, 5, 5])
```

Detailed Documentation

Documentation

Unflattens a tensor dim expanding it to a desired shape. For use with Sequential.

dim specifies the dimension of the input tensor to be unflattened, and it can be either int or str when Tensor or NamedTensor is used, respectively.

unflattened_size is the new shape of the unflattened dimension of the tensor and it can be a tuple of ints or a list of ints or torch.Size for Tensor input; a NamedShape (tuple of (name, size) tuples) for NamedTensor input.

Input: $(*, S_{\text{dim}}, *)$ ($*, S_{\text{dim}}, *$), where S_{dim} is the size at dimension dim and $*$ means any number of dimensions including none.

Output: $(*, U_1, \dots, U_n, *)$ ($*, U_1, \dots, U_n, *$), where $U_i = \text{unflattened_size}$ and $\prod_{i=1}^n U_i = S_{\text{dim}}$.

dim (Union[int, str]) – Dimension to be unflattened

unflattened_size (Union[torch.Size, Tuple, List, NamedShape]) – New shape of the unflattened dimension

alias of tuple[tuple[str, int]]

Returns the extra representation of the module.

Runs the forward pass.
